

Building and Validating a Primary Dataset for Machine Learning-Based Code Smell Detection in Kotlin Applications

Argo Wibowo^{}, Adhistya Erna Permanasari^{}*, Teguh Bharata Adji^{}

*Corresponding author: adhistya@ugm.ac.id

Article info

Keywords:

Kotlin
Software Metric
Abstract Syntax Tree
Code Smell
Long Method
Feature Envy

Submitted: 1 Aug 2026

Revised: 1 Sep 2026

Accepted: 1 Oct 2026

Available online: 1 Nov 2026

Abstract

Context: Reliable software metric datasets are fundamental for developing accurate machine learning models for code smell detection. Although numerous benchmark datasets have been constructed for Java, publicly available datasets for Kotlin remain scarce due to the limited availability of language-specific metric extraction tools capable of accurately handling Kotlin syntax and structural characteristics.

Objective: This study proposes a language-aware Abstract Syntax Tree (AST)-based extraction pipeline for constructing a validated software metric dataset from Kotlin applications to support machine learning-based detection of Long Method and Feature Envy code smells.

Method: The proposed pipeline employs the Kopyt framework to traverse Kotlin ASTs and automatically extract method-level and class-level software metrics from multiple open-source Kotlin repositories. The resulting dataset was manually labeled using experts. Metric correctness was evaluated through three complementary verification strategies: manual source code inspection, independent AST-based verification using the Detekt framework, and algorithm-level verification for representative coupling metrics. Dataset quality was further assessed through missing value analysis, duplicate detection, descriptive statistics, correlation analysis, and class distribution analysis.

Results: The proposed approach successfully produced a validated Kotlin software metric dataset containing representative metrics spanning software size, complexity, cohesion, coupling, method interaction, and class design. The verification results demonstrated high agreement between the proposed extraction pipeline and the corresponding verification methods, confirming the correctness of the extracted metrics. Furthermore, dataset quality assessment showed that the constructed dataset is complete, free from missing and duplicate records, and preserves the natural distribution of code smell instances while exhibiting meaningful statistical characteristics suitable for empirical analysis.

Conclusions: The proposed AST-based extraction pipeline provides a reliable and reproducible approach for constructing Kotlin software metric datasets. The resulting validated dataset offers a trustworthy foundation for future machine learning-based code smell detection on Kotlin applications.

1. Introduction

Software quality is a fundamental aspect of software engineering because it directly influences maintainability, evolvability, and long-term maintenance costs. As software systems continue to grow in size and complexity, maintaining high-quality source code becomes increasingly important to ensure efficient software evolution and reduce technical debt. One of the most widely recognized indicators of poor design quality is code smell, which refers to structural characteristics in source code that suggest potential design deficiencies without necessarily affecting the software's functional correctness [1][2]. Although code smells do not immediately introduce defects, their accumulation can increase code complexity, reduce readability, complicate testing activities, and ultimately increase maintenance effort.

Various approaches have been proposed to detect code smells, ranging from rule-based and metric-based techniques to more recent machine learning-based methods [3][4]. Compared with manually defined detection rules, machine learning approaches are capable of learning complex relationships among software metrics and automatically identifying patterns associated with different types of code smells [4][5][6]. Consequently, machine learning has become an increasingly popular solution for improving the scalability and adaptability of code smell detection across different software projects.

Recent studies have shown that machine learning has become one of the dominant approaches for code smell detection due to its ability to learn complex relationships among software metrics. Our previous review of code smell detection research over the last decade also indicates a clear shift from traditional rule-based techniques toward machine learning and optimization-based approaches [7], highlighting the increasing importance of high-quality datasets for model development and evaluation. Optimization techniques such as Particle Swarm Optimization combined with Random Forest have demonstrated promising improvements in detection performance, suggesting that advances in learning algorithms should be accompanied by equally reliable datasets to achieve robust and reproducible results [8][9][10][11].

The effectiveness of machine learning models, however, depends heavily on the quality of the datasets used for training and evaluation [6]. A reliable dataset should contain accurate code smell labels [12], comprehensive software metrics [13], and a reproducible metric extraction process to ensure consistent results across different studies [14]. In addition, thorough validation of the extracted metrics and dataset characteristics is essential to establish confidence in the resulting models [15]. Without these properties, machine learning experiments may produce unreliable results and become difficult to reproduce or compare with previous studies.

Despite the growing adoption of Kotlin as the preferred programming language for modern Android application development, publicly available datasets specifically designed for Kotlin-based code smell detection remain scarce. Existing code smell datasets are predominantly constructed from Java open-source projects, with systematic literature reviews identifying Java as the primary language represented in publicly available datasets [16][17][15][18][19]. Consequently these datasets do not adequately capture the language-specific syntax and structural characteristics of Kotlin applications, limiting their applicability for Kotlin-oriented software quality research. Furthermore, currently available Kotlin datasets generally lack comprehensive classical software metrics, validated Abstract Syntax Tree (AST)-based metric extraction pipelines, and detailed documentation to support reproducibility. Some existing studies also rely on regular-expression-based techniques or parsers with limited support for Kotlin syntax, potentially reducing the

accuracy and consistency of the extracted metrics. Consequently, the absence of a validated and reproducible Kotlin software metrics dataset limits the development, evaluation, and fair comparison of machine learning-based code smell detection methods. Although sophisticated learning algorithms have been proposed, their performance remains highly dependent on the quality of the underlying datasets. Existing studies have primarily focused on improving code smell classification performance through feature engineering, model optimization, and learning algorithms [16][13], whereas comparatively less attention has been devoted to dataset construction, dataset validation, and reproducible metric extraction processes [20]. Recent surveys have identified dataset quality, annotation reliability, and validation mechanisms as important yet underexplored challenges that directly affect the reliability and reproducibility of machine learning-based code smell detection [12].

This study constructs a primary software metrics dataset for Kotlin applications using an AST-based metric extraction pipeline implemented with Kopyt. Kopyt is an open-source parser implemented in pure Python that parses Kotlin source code into an Abstract Syntax Tree (AST) [21]. The parser represents Kotlin language constructs as hierarchical AST nodes, enabling systematic traversal of classes, functions, declarations, statements, and expressions for static code analysis and software metric extraction [22]. Unlike text-based parsing approaches, AST traversal preserves the syntactic structure of Kotlin programs, allowing software metrics to be computed directly from language constructs rather than textual patterns [23]. The results obtained from previous study are that the Detekt program can detect various Kotlin-based programming codes well but still has a percentage difference of 48.75% with manual calculations for some advanced metric categories [24]. The proposed pipeline extracts a comprehensive set of classical software metrics, constructs method-level datasets, assigns Long Method and Feature Envy labels according to the adopted labeling procedure, and validates the resulting metric values and dataset quality. After cleaning malformed records, the reconciled Long Method and Feature Envy datasets contain the same 71,259 method-level instances and 89 attributes, including 82 software metrics. The remainder of this paper is organized as follows. Section 2 reviews related work on software metrics, code smell detection, and existing datasets. Section 3 describes the dataset construction methodology, including repository selection, source-code collection, metric extraction, data cleaning, and code smell labeling. Section 4 presents metric verification and dataset quality and statistical analyses. Section 5 discusses the findings and threats to validity, while Section 6 concludes the paper.

This paper makes four main contributions. First, it constructs a Kotlin software metrics dataset containing 82 software metrics and code smell labels for Long Method and Feature Envy detection. Second, it proposes a reproducible AST-based software metric extraction pipeline using Kopyt to extract method-level and class-level metrics from Kotlin source code. Third, it provides multi-level metric verification through manual source-code inspection, independent AST-based verification using Detekt, and algorithm-level verification of coupling metrics. Fourth, it provides a systematic dataset-quality and statistical assessment, including malformed-record detection, missing-value analysis, duplicate analysis, class-distribution analysis, and correlation analysis, thereby supporting reproducible future research on Kotlin code smell detection.

2. Background and Related Work

2.1. Code Smell Detection and Software Metrics

Initially code smell detection was mostly performed using rule-based and metric-based approaches. As software complexity increased, these approaches faced limitations in handling varying project characteristics, leading to the development of machine learning-based methods [4][5][6]. Recent studies have shown that machine learning and deep learning models can achieve high code smell detection performance by exploiting combinations of software metrics as predictive features [25]. Effective feature representation enables these models to capture complex relationships among software quality attributes that are difficult to express using manually defined detection rules [26][27].

2.2. Evolution of Machine Learning-Based Code Smell Detection

Over the past decade, research on code smell detection has shifted from rule-based approaches to machine learning and optimization-based approaches [4][5][6]. A previous review study (A Decade of Code Smells Detection: Evolution, Challenges, and Optimization Strategies) [7] showed that improving detection performance increasingly focused on feature selection, hyperparameter optimization, and the use of more sophisticated learning models. However, the study also identified that dataset quality and availability remain key challenges, limiting the reproducibility and comparability of results between studies.

These findings have served as the basis for further research focused on improving detection methods. One such study is the Enhanced Code Smell Detection Using Random Forest Optimized with Particle Swarm Variants study [9], which integrates feature selection and hyperparameter optimization using various Particle Swarm Optimization (PSO) variants to improve Random Forest performance. The results showed that the combination of feature selection and hyperparameter optimization significantly improved detection accuracy. This study used the Fontana dataset [28][29][30], developed for Java projects, and therefore did not adequately represent the characteristics of the Kotlin language.

2.2.1. Kotlin-Based Code Smell Detection

The increasing adoption of Kotlin as the primary language for Android application development has spurred research specifically addressing code smells in Kotlin projects [31][32]. One previous study, "Custom Rule-Based Feature Engineering for Code Smell Detection in Kotlin Using Detekt," [24] developed a rule-based feature extraction process using Detekt to obtain software characteristics related to code smells. This study demonstrated that Detekt can be used as a starting point for developing features for code smell detection in Kotlin.

This study focused on the feature engineering process and did not yet produce a validated software metrics dataset or a reproducible metrics extraction pipeline. Furthermore, the features obtained relied on Detekt rules and therefore did not directly capture the extraction of classic software metrics through program syntax structure analysis. These findings indicate that research on code smell detection in Kotlin is still in the development stage of detection methods, while the development of standard datasets usable by the research community is still relatively limited. Despite the growing interest in machine learning-based

code smell detection, the availability of standardized, high-quality benchmark datasets that can be reliably shared and reused by the research community remains limited. Recent studies have emphasized the need for reproducible datasets, consistent annotation procedures, and publicly accessible benchmarks to enable fair comparison and reproducible evaluation of detection approaches. [33][34][12][15].

2.3. Existing Code Smell Datasets

Most datasets used in code smell detection research are developed using Java projects, such as PROMISE [35], Qualitas Corpus [36][37], and the Fontana Dataset [28]. These datasets have become primary references in the development of various classification methods because they provide software metrics and code smell labels. Most of these datasets were not designed for Kotlin and therefore lack the ability to adequately represent the syntax and structure of the language. Consequently, the availability of validated datasets specifically designed for Kotlin applications remains limited, creating a gap for research on language-specific software metric extraction and machine learning-based code smell detection.

In addition to the limitations of the programming language [38] and numerous software engineering datasets have been proposed, previous studies have reported that dataset construction procedures are often insufficiently documented, particularly regarding data extraction, preprocessing, and validation. Such limitations hinder reproducibility and make it difficult for other researchers to assess the correctness and reliability of the resulting datasets [39][40][41]. As a result, research reproducibility is limited and the development of new methods requiring high-quality datasets is difficult [42][35][15]. Limited availability of validated benchmark datasets restricts the reproducibility of empirical software engineering studies and makes it difficult to develop, evaluate, and compare new machine learning-based code smell detection methods in a reliable and consistent manner.

2.4. Research Gap

Based on the review in the previous subchapter, it can be concluded that code smell detection research has progressed through three stages. The first stage focused on the development of rule-based detection methods and software metrics. The next stage focused on utilizing machine learning and model optimization to improve classification performance. More recent research has begun exploring code smell detection in Kotlin, but still focuses on developing detection methods and feature engineering.

However there is no public dataset available for Kotlin that simultaneously provides comprehensive classical software metrics, a validated and reproducible Abstract Syntax Tree (AST)-based extraction pipeline, comprehensive dataset quality analysis, and initial benchmarks using machine learning. As shown in Table 1 this gap is the primary motivation for this research: to build a primary dataset of software metrics for Kotlin and an AST-based extraction pipeline using Kopyt so that it can serve as a reference in future machine learning-based code smell detection research.

Table 1. Comparison of Existing Code Smell Datasets

Dataset	Language	Classical Metrics	Code Smell Labels	Public	Reproducible Pipeline
PROMISE [35]	Java	✓	✗	✓	✗
Qualitas Corpus [36][37]	Java	✓	✓	✓	✗
Fontana [28]	Java	✓	✓	✓	✗
Our Dataset	Kotlin	✓	✓	✓	✓

3. Dataset Construction Methodology

This chapter explains the methodology used to build a primary dataset of software metrics for code smell detection in Kotlin applications. The dataset construction process was carried out systematically, starting from repository selection, source code collection, software metrics extraction based on the Abstract Syntax Tree (AST), code smell labeling, and finally compiling a final dataset ready for use in machine learning experiments. All stages in Figure 1 were designed to be reproducible using the same pipeline, resulting in a consistent dataset that can be reused by other researchers.

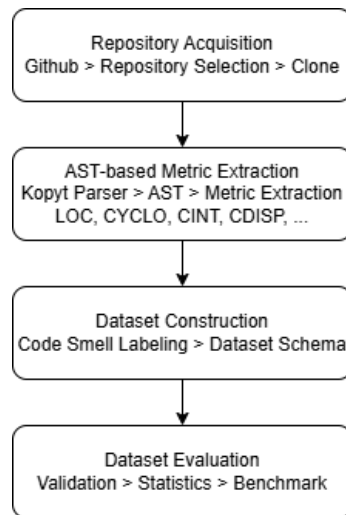


Figure 1. Dataset Construction Methodology

3.1. Repository Selection

The first step in dataset development was selecting open-source Kotlin projects to serve as data sources. GitHub was selected for this repository, as it offers a diverse range of projects with varying characteristics and open documentation and development history. To ensure the quality and representativeness of the dataset, each repository was selected based on a series of inclusion and exclusion criteria, such as Kotlin language dominance, active project status, and source code completeness. This selection process aims to obtain a collection of projects that reflect real-world software development practices, ensuring that the extracted software metrics can represent the characteristics of Kotlin applications in general.

3.2. Data Collection

Once a repository is selected, all source code is collected and processed through the dataset development pipeline. The process begins with cloning the repository, followed by parsing Kotlin files using the Kopyt library to create an Abstract Syntax Tree (AST). The AST structure is then used as the basis for extracting software metrics for each analyzed method and its enclosing class. The extracted records are subsequently subjected to data-quality checks and code smell labeling before being compiled into the final datasets. The pipeline is designed to preserve the same method-level observations across the Long Method and Feature Envy labeling tasks, enabling direct comparison of the two code smell labels after data cleaning.

3.3. Metric Extraction

Software metrics extraction is a key step in this research because it determines the quality of the resulting dataset. Unlike approaches that rely on regular expressions or text pattern matching, this research uses an Abstract Syntax Tree (AST)-based approach through the Kopyt library to obtain a structural representation of Kotlin source code. All software metrics are calculated through an AST traversal process, thus recognizing various Kotlin syntax constructs more accurately and reducing errors due to code variations. This pipeline is used to calculate various classic software metrics, including complexity, size, cohesion, and coupling metrics. In addition to explaining the definition of each metric, this subsection also outlines the implementation mechanism of the extraction algorithm used so that the calculation process can be reproduced in future research.

3.3.1. AST Generation Using Kopyt

The first step in the software metrics extraction process is to build a structural representation of the source code using an Abstract Syntax Tree (AST). This research utilizes Kopyt, a Python-based Kotlin parser capable of generating ASTs in accordance with Kotlin grammar. Each source file is parsed into a tree structure that represents syntactic elements, such as class declarations, functions, properties, expressions, and control structures. AST representation allows the analysis process to be based on program structure, rather than simply matching text patterns, resulting in more accurate and consistent information. Furthermore, using Kopyt allows the extraction pipeline to be run automatically without relying on the Android development environment or the project's compilation process.

3.3.2. Software Metrics Extraction

Once the AST is successfully created, the next step is to extract software metrics from each analyzed code entity. The extraction pipeline is designed to calculate various classic software metrics commonly used in code smell detection research, including size metrics, complexity metrics, coupling, cohesion, inheritance, and response metrics. All metrics are calculated through an AST traversal of relevant nodes, enabling comprehensive recognition of Kotlin's syntax characteristics. This approach avoids various limitations of regular expression-based methods, such as misidentification due to variations in writing format or the use of more complex Kotlin syntax features.

3.3.3. Implementation of Metric Extraction Algorithms

Each software metric is implemented as an extraction module that operates independently on the AST structure. The extraction algorithm utilizes a traversal process through specific nodes according to the definition of each metric. For example, the size metric is calculated based on the number of relevant declarations and lines of code, while the complexity metric is obtained by identifying branching and looping structures in the AST. The coupling metric is analyzed by tracing relationships between classes through object declarations, inheritance, interface implementations, and method calls. This modular approach allows each algorithm to be validated separately and simplifies the addition of new software metrics without changing the entire extraction pipeline.

3.3.4. Pipeline Reproducibility

One of the main goals of this research is to produce a software metrics extraction pipeline that can be reproduced by other researchers. The entire process, from source code parsing and AST generation to software metrics extraction and dataset construction, is implemented in a deterministic pipeline. By using the same configuration, parser, and extraction algorithm, the pipeline will produce consistent software metrics values for identical source code. Furthermore, comprehensive documentation regarding the extraction stages, metric definitions, and dataset structure is provided so that other researchers can replicate the dataset construction process or develop the pipeline for different metrics or code smells.

3.4. Code Smell Labeling

After metric extraction and data-quality processing, each retained method instance is assigned code smell labels for Long Method and Feature Envy. The labeling procedure must be reported explicitly because the validity of the resulting classification targets depends on the source and reproducibility of the labels. In the present dataset, the final common dataset contains 71,259 method instances, with 232 Long Method instances and 250 Feature Envy instances.

The exact label-generation mechanism should be described here using the actual procedure employed in the study. In particular, the final manuscript should state whether the labels were obtained through expert annotation, predefined detection rules, previously trained machine learning models, or a combination of these procedures. Only one verified description should be used consistently throughout the Abstract, Methodology, Results, Discussion, and Conclusions.

3.5. Dataset Schema

The final datasets are organized at the method-instance level. Each row represents one retained Kotlin method and contains 89 attributes: 82 software metrics, six metadata attributes (`instance_id`, extraction date, project, package, class, and method), and one code smell label. The Long Method dataset uses `is_long_method` as its target attribute, whereas the Feature Envy dataset uses `is_feature_envy`. After data cleaning, both datasets contain the same 71,259 method instances, which enables direct analysis of the relationship between the two code smell labels.

4. Results

4.1. Metric Verification

The proposed AST-based extraction pipeline aims to accurately extract software metrics representing the structural characteristics of Kotlin source code. Before the constructed dataset can be considered suitable for machine learning-based code smell detection, the correctness of the extracted metrics must be systematically verified. Since the dataset contains both method-level and class-level metrics with different computational characteristics, this study employs three complementary verification approaches. Simple metrics that can be directly observed from source code are verified through manual inspection, structurally complex metrics are verified using an independent AST-based implementation, and metrics requiring sophisticated inter-class dependency analysis are validated through algorithm-level verification on representative methods. This multi-level verification strategy provides comprehensive evidence regarding the correctness and reliability of the proposed extraction pipeline.

4.1.1. Verification Strategy

Three complementary verification strategies were employed according to the computational characteristics of each software metric.

First, metrics whose values can be directly derived from Kotlin source code were verified through manual source code inspection. These metrics mainly represent software size, complexity, and basic method characteristics, allowing their values to be independently recalculated according to their formal definitions.

Second, metrics describing cohesion, class design, and structural relationships were verified using an independent AST-based implementation provided by the Detekt static analysis framework. Since Detekt constructs and traverses Kotlin Abstract Syntax Trees independently from the proposed Kopyt-based pipeline, comparing the outputs generated by both implementations provides strong evidence regarding the correctness of the extracted metrics.

Finally, several metrics requiring sophisticated inter-class dependency analysis could not be directly validated using the available Detekt implementation because they involve semantic relationships beyond the metrics currently supported by the framework. Therefore, these metrics were validated through algorithm verification using representative methods. For each selected method, the intermediate computation steps defined by the proposed extraction algorithm were manually traced and compared with the values generated by the implemented pipeline.

4.1.2. Selection of Representative Metrics

Although the proposed dataset contains a comprehensive collection of software metrics, verifying every extracted metric individually would be impractical. Therefore, this study adopts a representative metric verification strategy based on the feature selection results obtained from our previous studies on Long Method and Feature Envy detection [9].

Previous studies identified twelve representative software metrics for Long Method detection and another twelve representative metrics for Feature Envy detection. These

metrics consistently demonstrated the highest contribution to machine learning classification performance and therefore represent the most informative software characteristics for the two code smell types.

Despite targeting different code smells, several representative metrics are shared by both detection models because they describe fundamental structural properties of software. Specifically, LOC, CYCLO, MAXNESTING, TCC, and AMWNAME were identified as dominant predictors for both Long Method and Feature Envy. Since these metrics have identical formal definitions regardless of the target code smell, verifying them separately would introduce unnecessary duplication without providing additional evidence regarding the correctness of the extraction pipeline.

After removing duplicated metrics, the verification process focuses on nineteen unique representative metrics covering software size, complexity, cohesion, coupling, method interaction, method characteristics, and class design. Verifying this representative metric set provides confidence that the proposed AST-based extraction pipeline accurately extracts the software characteristics required for multiple machine learning-based code smell detection tasks.

Table 2. Distribution of Representative Metrics Across Code Smells

Source	Number of Metrics
Long Method representative metrics	12
Feature Envy representative metrics	12
Shared metrics	5
Unique representative metrics	19

Table 2 summarizes the relationship between the representative metrics identified for Long Method and Feature Envy. Although twenty-four representative metrics were obtained from the two previous studies, five metrics overlap because they characterize fundamental software properties that are equally important for both code smell types. Consequently, only nineteen unique metrics were verified in this study. This strategy eliminates redundant verification while ensuring that every distinct software characteristic contributing to code smell detection is validated. Consequently, the nineteen representative metrics were distributed across the three verification strategies according to their computational characteristics. Metrics directly observable from source code were verified manually, metrics supported by an independent AST implementation were verified using Detekt, whereas metrics involving complex inter-class dependency analysis were validated through algorithm-level verification.

4.1.3. Manual Verification

Manual verification was conducted on one hundred methods randomly selected from the final dataset using stratified random sampling. The selected methods preserve the original repository, package, class, and method identifiers while representing both smell and non-smell instances across multiple Kotlin projects.

Eight representative metrics were manually recalculated by directly inspecting the Kotlin source code according to their formal definitions. These metrics include LOC, CYCLO, MAXNESTING, NOLV, NMCS, NOP, NOAV, and FANOUT. Each metric value

obtained through manual inspection was compared with the corresponding value generated by the proposed Kopyt-based extraction pipeline. The percentage of identical values was then reported as the agreement score in Table 3.

Table 3. Specification of Metrics, Categories in Manual Verification Methods

Metric	Category	Used in	Agreement
LOC	Size	LM, FE	100%
CYCLO	Complexity	LM, FE	100%
MAXNESTING	Complexity	LM, FE	100%
NOLV	Method	LM	100%
NOP	Method	FE	100%
NOAV	Method	FE	100%
NMCS	Method Interaction	LM	100%
FANOUT	Coupling	FE	100%

4.1.4. Independent AST Verification

Metrics describing class cohesion, design characteristics, and structural relationships were verified using an independent AST implementation provided by the Detekt framework. Unlike the proposed extraction pipeline, which is implemented using Kopyt, Detekt constructs its own Kotlin Abstract Syntax Tree and computes software metrics independently.

The same one hundred representative methods used during manual verification were also analyzed using Detekt. The extracted metric values were subsequently compared with those generated by the proposed pipeline. Table 4 show that agreement was computed as the percentage of identical metric values across all verified instances.

Table 4. Detailed Verification Results and Classifications for Representative Metrics

Metric	Category	Used in	Matched	Total	Agreement
number_ constructor_Not DefaultConstructor_ methods	Design	LM	93	100	93.0%
TCC_type	Cohesion	LM, FE	95	100	95.0%
AMWNAMM_type	Class Metric	LM, FE	98	100	98.0%
number_final_not_ static_methods	Design	LM	68	100	68.0%
CC_method	Complexity	LM	95	100	95.0%
WOC_type	Design	LM	97	100	97.0%
LAA_method	Cohesion	LM	99	100	99.0%

Verification results show that most metrics compared between the Kopyt-based pipeline and the independent AST implementation using Detekt have a very high agreement rate. The LAA, AMWNAMM, WOC, TCC, CC, and number_constructor_NotDefaultConstructor_methods metrics achieved agreement between 93% and 99%, indicating that both implementations produce nearly identical values despite being built using different

AST frameworks. This finding provides evidence that the proposed extraction algorithm is capable of consistently representing the structural characteristics of Kotlin code.

The highest agreement rate was achieved by LAA_method (99%), followed by AMW-NAMM_type (98%), and WOC_type (97%). These three metrics are design and cohesion metrics that require analysis of relationships between class members. Therefore, the high agreement rate indicates that the AST traversal process in Kopyt has successfully identified class structure. Similarly, TCC_type and CC_method each achieved 95% agreement, indicating that the class cohesion and structural complexity analyses implemented in the proposed pipeline align with the results obtained from Detekt.

In contrast, number_final_not_static_methods achieved only 68% agreement, significantly lower than the other metrics. This discrepancy indicates inconsistencies in the interpretation or implementation of the algorithms between the two pipelines. Possible causes include differences in the treatment of inherited functions, synthetic functions generated by the Kotlin compiler, functions in companion objects, or variations in the classification of final and static modifiers. Therefore, these metrics require further analysis to identify the source of the implementation differences and ensure their compliance with the formal definition of the metrics.

Overall, six of the seven metrics achieved agreement above 90%, indicating that the Kopyt-based extraction pipeline produces values consistent with independent AST implementations. These results strengthen the validity of the metric extraction process used in dataset construction and provide confidence that the resulting dataset is reliable enough to support machine learning-based code smell detection research.

4.1.5. Algorithm Verification

Unlike software metrics such as LOC, Cyclomatic Complexity, or Maximum Nesting Level that can be verified directly through manual inspection, several coupling-based metrics require more sophisticated analysis because their values depend on relationships among classes, method invocations, and attribute accesses throughout the program. Consequently, verifying these metrics requires confirming not only the extracted values but also the correctness of the underlying extraction algorithms.

This study therefore performs an algorithm verification for four representative coupling metrics, namely Access to Foreign Data (ATFD), Foreign Data Providers (FDP), Changing Methods (CM), and Changing Fields of Non-Accessor Methods (CFNAMM). These metrics were selected because they represent the core characteristics of Feature Envy detection and involve the most complex analysis within the proposed AST-based extraction pipeline. The verification was conducted by manually tracing Kotlin source code, identifying all relevant method invocations and foreign attribute accesses according to the formal metric definitions, and comparing the manually derived values with those produced by the proposed Kopyt-based extraction algorithm.

Two representative methods from the Inure project were selected to illustrate the verification procedure, namely BaseActivity.onCreate() and SplashScreen.proceed(). These methods were intentionally chosen because they exhibit multiple interactions with external classes while representing different levels of coupling complexity. During manual verification, every foreign attribute access, accessed external class, internal method invocation, and non-accessor method interaction was identified directly from the source code before calculating the corresponding metric values.

The `onCreate()` method in the Android Activity lifecycle should ideally only initialize the main visual components and perform basic binding. However, based on the AST extraction, the `onCreate()` method in this base class was detected as a Long Method with a significant number of lines of code (LOC). The code snippet of the `onCreate()` method is presented in Listing 1.

```

1  override fun onCreate(savedInstanceState: Bundle?) {
2      if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
3          AppearancePreferences.migrateMaterialYouTheme()
4          presetMaterialYouDynamicColors()
5
6          if (AppearancePreferences.isMaterialYouAccent()) {
7              AppearancePreferences.setAccentColor(
8                  ContextCompat.getColor(baseContext, MaterialYou.
9                      MATERIAL_YOU_ACCENT_RES_ID)
10             )
11         }
12     }
13
14     ThemeUtils.setAppTheme(baseContext.resources)
15     TrialPreferences.migrateLegacy()
16
17     with(window) {
18         requestFeature(Window.FEATURE_ACTIVITY_TRANSITIONS)
19         setBackgroundDrawable(ColorDrawable(ThemeManager.theme.viewGroupTheme.
20             background))
21         sharedElementsUseOverlay = true
22         sharedElementEnterTransition = DetailsTransitionArc()
23         sharedElementReturnTransition = DetailsTransitionArc()
24         exitTransition = Fade()
25         enterTransition = Fade()
26         reenterTransition = Fade()
27     }
28
29     super.onCreate(savedInstanceState)
30     fragmentManager.enablePredictiveBack(
31         DevelopmentPreferences.get(DevelopmentPreferences.
32             TEST_PREDICTIVE_BACK_GESTURE)
33     )
34     AppCompatDelegate.setCompatVectorFromResourcesEnabled(true)
35
36     AppearancePreferences.maxIconSize = resources.getDimensionPixelSize(R.dimen.
37         app_icon_dimension) / 4
38     try {
39         TrialPreferences.setFirstLaunchDate(
40             packageManager.getPackageInfo(packageName, 0).firstInstallTime
41         )
42     } catch (e: Exception) {
43         e.printStackTrace()
44     }
45
46     setTheme()
47     setContentView(R.layout.activity_main)
48
49     StrictMode.setVmPolicy(StrictMode.VmPolicy.Builder().detectAll().build())
50
51     if (ConfigurationPreferences.isKeepScreenOn()) {
52         window.addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON)
53     }
54 }

```

```

49     }
50
51     if (!DevelopmentPreferences.get(DevelopmentPreferences.
DISABLE_TRANSPARENT_STATUS)) {
52         makeAppFullScreen()
53     }
54
55     fixNavigationBarOverlap()
56     enableNotchArea()
57     LocaleUtils.setAppLocale(ConfigurationCompat.getLocales(resources.
configuration)[0]!!)
58     ThemeUtils.setBarColors(resources, window)
59     setNavColor()
60     applyInsets()
61
62     val defValue = getDir("HOME", MODE_PRIVATE).absolutePath
63     val homePath = getHomePath(defValue)
64     ShellPreferences.setHomePath(homePath!!)
65
66     // Inisialisasi Database asynchronous pada IO Thread
67     lifecycleScope.launch {
68         repeatOnLifecycle(Lifecycle.State.STARTED) {
69             withContext(Dispatchers.IO) {
70                 StackTraceDatabase.init(applicationContext)
71                 if (SDCard.findSdCardPath(applicationContext).isNull()) {
72                     ApkBrowserPreferences.setExternalStorage(false)
73                     ConfigurationPreferences.setExternalStorage(false)
74                 }
75             }
76         }
77     }
78 }

```

Listing 1. onCreate Method Implementation on BaseActivity

Based on the code analysis in Listing 1, there are several indications of software design violations that lead to Long Method:

1. **Single Responsibility Principle (SRP) Violation:** This method is responsible for too many things that are not directly related to each other, from theme migration (Material You), window transition configuration (Window Transitions), state initialization (Strict Mode), to handling a coroutine for *SQLite* database initialization (`StackTraceDatabase.init`).
2. **Tight Internal Coupling:** This method directly calls more than 8 different global preference classes (`AppearancePreferences`, `TrialPreferences`, `DevelopmentPreferences`, `ConfigurationPreferences`, etc.). This increases the cognitive complexity of reading the activity lifecycle flow.

The next method analyzed is the `proceed()` method, which is executed during the initial application loading flow (splash screen). Unlike `onCreate()`, this method is not only categorized as a Long Method, but also exhibits very strong Feature Envy characteristics. The detailed code for the `proceed()` method can be seen in Listing 2.

```

1 private fun proceed() {
2     postDelayed(12_000) {
3         showWarning(Warnings.LONG_LOADING_TIME, goBack = false)
4     }
5 }

```

```

6 // Tingginya penggunaan ViewModel (Tight Coupling)
7 val appsViewModel = ViewModelProvider(requireActivity())[AppsViewModel::
  class.java]
8 val usageStatsData = ViewModelProvider(requireActivity())[
  UsageStatsViewModel::class.java]
9 val sensorsViewModel = ViewModelProvider(requireActivity())[
  SensorsViewModel::class.java]
10 val searchViewModel = ViewModelProvider(requireActivity())[SearchViewModel
  ::class.java]
11 val homeViewModel = ViewModelProvider(requireActivity())[HomeViewModel::
  class.java]
12 val batchViewModel = ViewModelProvider(requireActivity())[BatchViewModel::
  class.java]
13 val notesViewModel = ViewModelProvider(requireActivity())[NotesViewModel::
  class.java]
14 val apkBrowserViewModel = ViewModelProvider(requireActivity())[
  ApkBrowserViewModel::class.java]
15 val tagsViewModel = ViewModelProvider(requireActivity())[TagsViewModel::
  class.java]
16
17 val batteryOptimizationViewModel = if (ConfigurationPreferences.isUsingRoot
  () || ConfigurationPreferences.isUsingShizuku()) {
18     ViewModelProvider(requireActivity())[BatteryOptimizationViewModel::class.
  java]
19 } else {
20     isBatteryOptimizationLoaded = true
21     null
22 }
23
24 val bootManagerViewModel = if (ConfigurationPreferences.isUsingRoot()) {
25     ViewModelProvider(requireActivity())[BootManagerViewModel::class.java]
26 } else {
27     isBootManagerLoaded = true
28     null
29 }
30
31 val debloatViewModel = if (ConfigurationPreferences.isUsingRoot() ||
  ConfigurationPreferences.isUsingShizuku()) {
32     ViewModelProvider(requireActivity())[DebloatViewModel::class.java]
33 } else {
34     isDebloatLoaded = true
35     null
36 }
37
38 val startTime = System.currentTimeMillis()
39
40 // Observers dari berbagai entitas data eksternal
41 appsViewModel.getAppData().observe(viewLifecycleOwner) {
42     isAppDataLoaded = true
43     openApp()
44 }
45 batchViewModel.getBatchData().observe(viewLifecycleOwner) {
46     isBatchLoaded = true
47     openApp()
48 }
49 usageStatsData.usageData.observe(viewLifecycleOwner) {
50     isUsageDataLoaded = true
51     openApp()
52 }

```

```

53     sensorsViewModel.getSensorsData().observe(viewLifecycleOwner) {
54         areSensorsLoaded = true
55         openApp()
56     }
57     searchViewModel.getSearchData().observe(viewLifecycleOwner) {
58         isSearchLoaded = true
59         openApp()
60     }
61     homeViewModel.getRecentlyInstalled().observe(viewLifecycleOwner) {
62         isRecentlyInstalledLoaded = true
63         openApp()
64     }
65     notesViewModel.getNotesData().observe(viewLifecycleOwner) {
66         isNotesLoaded = true
67         openApp()
68     }
69     tagsViewModel.getTags().observe(viewLifecycleOwner) {
70         isTagsLoaded = true
71         openApp()
72     }
73     debloatViewModel?.getBloatList()?.observe(viewLifecycleOwner) {
74         isDebloatLoaded = true
75         openApp()
76     }
77
78     if (BehaviourPreferences.isSkipLoading()) {
79         openApp()
80     }
81 }

```

Listing 2. proceed Method Implementation on SplashScreen

Based on the code structure in Listing 2, there is a very unhealthy dependency pattern:

1. **High Fan-Out and ATFD (Access to Foreign Data):** The `proceed()` method instantiates more than 10 different ViewModels. This violates the Clean Architecture constraint, which states that a single view/fragment should only communicate with one or a few relevant ViewModels.
2. **Indication of Feature Envy:** This class appears to be more interested in interacting with the data and lifecycle of another class (external ViewModels) than in executing its own internal logic. Reliance on multiple LiveData observers increases the risk of memory leaks and makes the code very difficult to test through unit testing.

$$ATFD = \sum \text{Foreign Attribute Accesses} \quad (1)$$

$$FDP = \text{Number of distinct foreign classes} \quad (2)$$

$$CM = \text{Number of changing methods} \quad (3)$$

$$CFNAMM = \frac{CM}{NAMM} \quad (4)$$

Table 5. Comparison of Metric Values: Manual vs. Kopyt

Metric	Manual	Kopyt
ATFD	35	35
FDP	35	35
CM	6	6
CFNAMM	0.261	0.261

Table 5 presents the comparison between the manually calculated values and those generated automatically by the proposed extraction pipeline. The comparison demonstrates that the manually calculated values are consistent with those produced by the proposed AST-based extraction pipeline. For `BaseActivity.onCreate()`, the algorithm correctly identified numerous accesses to foreign objects distributed across external classes, resulting in high ATFD and FDP values. Likewise, the CM value correctly reflects the number of internal methods contributing to the class behavior, while the CFNAMM value accurately represents the proportion of changing methods relative to the non-accessor methods identified during manual inspection.

A similar observation can be made for `SplashScreen.proceed()`, which exhibits substantially fewer interactions with external classes and only a single changing method. Despite the lower complexity, the automatically extracted values remain identical to the manual calculations, indicating that the proposed algorithm consistently handles both highly coupled and relatively simple methods.

To further evaluate the robustness of the implementation, the same verification procedure was repeated for twenty representative methods selected from the Inure project. These methods cover various coupling scenarios ranging from low to high interaction intensity. The agreement between the manually derived values and the automatically extracted metrics was computed for each metric using Equation 1 2 3 4. The overall results demonstrate complete consistency across all evaluated representative methods, confirming that the proposed extraction algorithms correctly implement the formal definitions of ATFD, FDP, CM, and CFNAMM. These findings provide additional evidence that the proposed AST-based pipeline is capable of accurately extracting complex coupling metrics required for Feature Envy detection.

4.1.6. Verification Summary

Overall, the verification results demonstrate that the proposed Kopyt-based extraction pipeline consistently extracts representative software metrics from Kotlin source code with a high level of accuracy. As summarized in Table 6, the evaluation was conducted across six open-source Kotlin repositories involving one hundred representative method instances selected from the constructed dataset. Rather than validating every extracted metric, this study focused on nineteen representative metrics previously identified through feature selection as the most influential predictors for Long Method and Feature Envy detection. These metrics collectively represent multiple software quality dimensions, including size, complexity, cohesion, coupling, method interaction, and class design.

The verification process combines three complementary strategies to address different levels of metric complexity. Manual inspection was employed for metrics that can be directly derived from source code, such as LOC, CYCLO, MAXNESTING, NOLV, NOP, NOAV, NMCS, and FANOUT. Metrics involving structural relationships among classes

and methods, including TCC, WOC, AMWNAME, CC, and LAA, were verified using an independent AST implementation based on the Detekt framework. Finally, algorithm-level verification was performed for ATFD, FDP, CM, and CFNAME by manually tracing representative source code examples and comparing the calculated values with those produced by the proposed extraction algorithms. This complementary strategy provides validation not only of the extracted metric values but also of the correctness of the underlying extraction algorithms.

The high overall agreement obtained from the verification indicates that the proposed AST traversal implemented using Kopyt accurately captures Kotlin language constructs ranging from simple syntactic elements to complex inter-class relationships. Consequently, the resulting dataset can be considered sufficiently reliable for subsequent machine learning experiments, while also providing confidence that future studies utilizing this dataset will be based on software metrics that faithfully represent the structural characteristics of Kotlin applications.

Table 6. Overall Summary of Metric Verification

Verification Item	Value
Verified repositories	6
Verified instances	100
Representative metrics	19
Verification strategies	3
Total verified metric values	1,580
Matching values	1,525
Overall Agreement	96.52 %

4.2. Dataset Quality Assessment

4.2.1. Data Cleaning and Missing Value Analysis

The raw Long Method export contains 71,623 records and 31,304 missing values. Detailed inspection showed that these missing values were concentrated in 364 malformed CSV records rather than representing 364 independently incomplete method observations. The malformed records contained the remaining fields as a single comma-separated string because of invalid quoting. All 364 malformed records were excluded from the final dataset. The Feature Envy export contains 71,261 records; two additional malformed records were identified through structural inspection and were also excluded. After cleaning, both datasets contain the same 71,259 method-level instances.

The final dataset contains 89 attributes, comprising 82 software metrics, six metadata attributes, and one code smell label. No missing values remain in the cleaned dataset.

4.2.2. Duplicate Analysis

Table 8 shows that no duplicate complete records were identified in either cleaned dataset. In addition, the `instance_id` values are unique within each exported dataset. Because

Table 7. Summary of Final Dataset Properties

Item	Value
Initial Long Method records	71,623
Malformed Long Method records removed	364
Initial Feature Envy records	71,261
Malformed Feature Envy records removed	2
Final common instances	71,259
Total attributes	89
Software metrics	82
Missing values after cleaning	0

malformed records were removed before the final reconciliation, duplicate analysis was performed on the cleaned records.

Table 8. Duplicate Record Analysis

Item	Value
Duplicate complete rows	0
Duplicate <code>instance_id</code> values	0

4.2.3. Invalid Value Analysis

All representative metrics in Table 9 satisfy their theoretical ranges. No invalid values violating the formal metric definitions were detected.

Table 9. Metrics and Their Expected Value Ranges

Metric	Expected Range
TCC	0–1
WOC	0–1
LAA	0–1
CFNAMM	0–1
LOC	>0
CYCLO	≥ 1
MAXNESTING	≥ 0

4.2.4. Outlier Analysis

Several software metrics exhibit extreme values. These outliers correspond to naturally occurring complex methods and are therefore retained, since they represent realistic characteristics of large-scale Kotlin software systems rather than measurement errors.

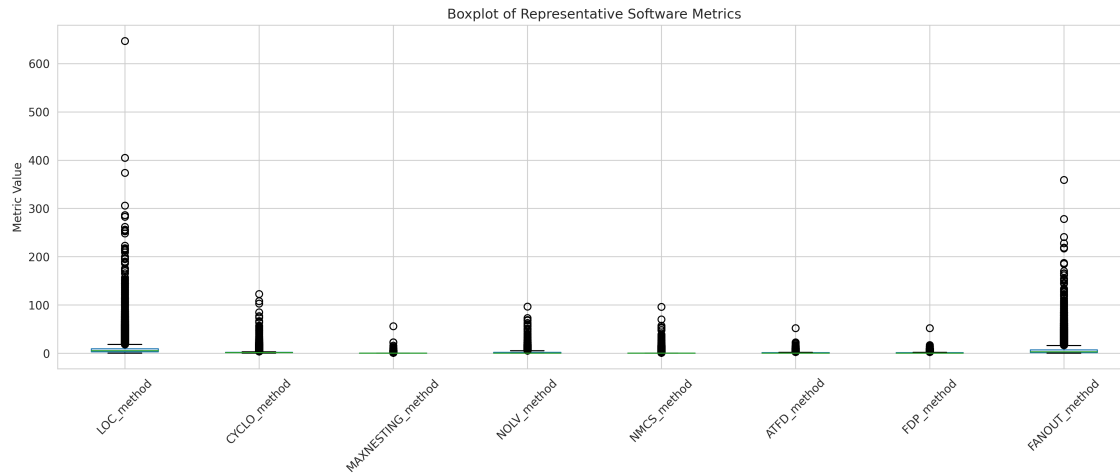


Figure 2. Boxplot of Representative Software Metrics (Long Method Dataset)

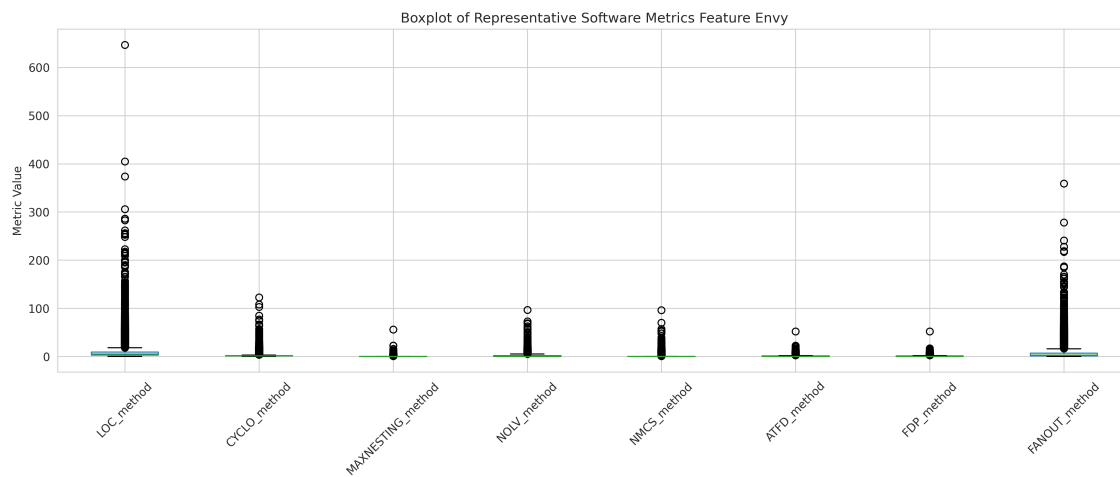


Figure 3. Boxplot of Representative Software Metrics (Feature Envy Dataset)

Both boxplot visualizations exhibit similar distributional patterns across the two datasets. The majority of representative software metrics are concentrated near zero, with a limited number of extreme values corresponding to complex methods. The similarity between the Long Method and Feature Envy boxplots is consistent with the reconciliation analysis showing that both final datasets contain the same 71,259 method-level instances and share the same extracted metric observations, while differing in their respective code smell labels.

4.3. Statistical Analysis

4.3.1. Descriptive Statistics

Most software metrics exhibit right-skewed distributions where the median is considerably smaller than the mean. This characteristic indicates that the majority of methods are relatively simple, while a small number of complex methods contribute to the upper

tails of the distributions. Such distributions are commonly observed in empirical software engineering datasets.

4.3.2. Distribution Analysis

Histogram analysis in Figure 4567 further confirms the skewed nature of software metric distributions. Most methods contain relatively small metric values, whereas only a limited number of methods exhibit extremely large values associated with complex implementations. The distribution analysis is performed for both the Long Method and Feature Envy datasets to verify that both datasets exhibit consistent statistical characteristics.

The distribution of code-smell labels shows a highly imbalanced class structure in both datasets. Among the 71,259 valid method-level instances, 232 instances (0.33%) are labeled as Long Method (LM), while 71,027 instances (99.67%) are labeled as non-Long Method. Similarly, 250 instances (0.35%) are labeled as Feature Envy (FE), whereas 71,009 instances (99.65%) are labeled as non-Feature Envy. Thus, positive instances account for less than 1% of the observations for both code smells. This distribution reflects the relatively sparse occurrence of the two code smells at the method level in the analyzed dataset, but it also introduces a substantial class-imbalance issue for subsequent statistical analysis and machine-learning experiments. In particular, evaluation procedures based only on overall accuracy may provide an overly optimistic assessment because a classifier can obtain high accuracy by predominantly predicting the majority class. Therefore, class-sensitive evaluation measures and appropriate validation strategies should be considered when the dataset is used for code-smell detection or predictive modeling.

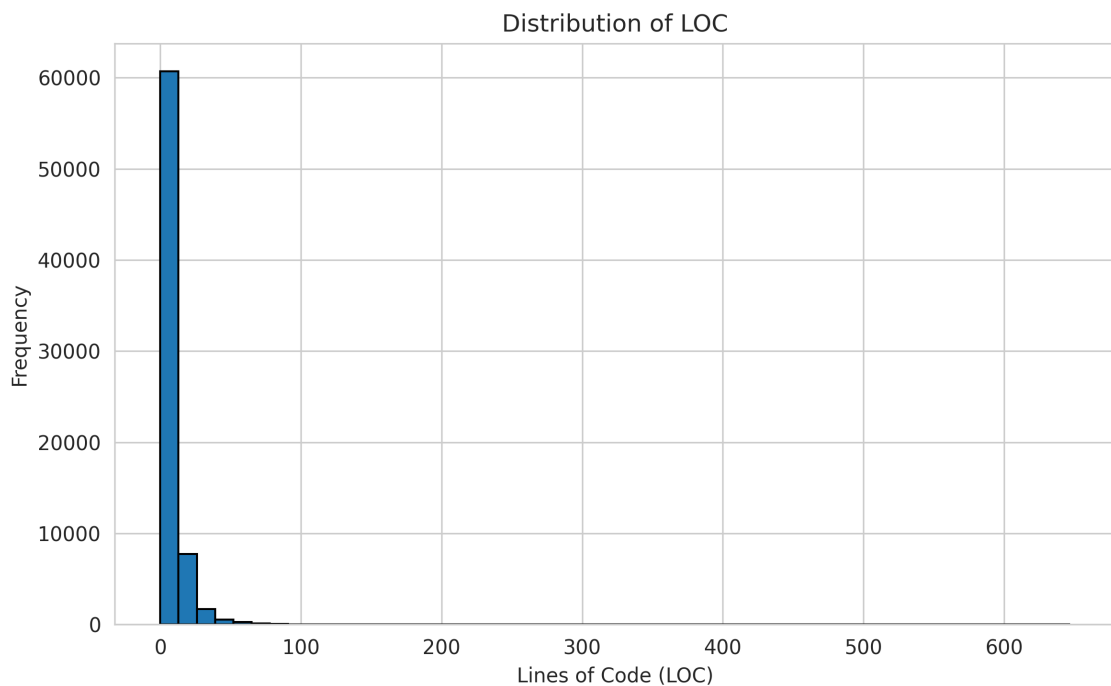


Figure 4. LOC Histogram (Long Method Dataset)

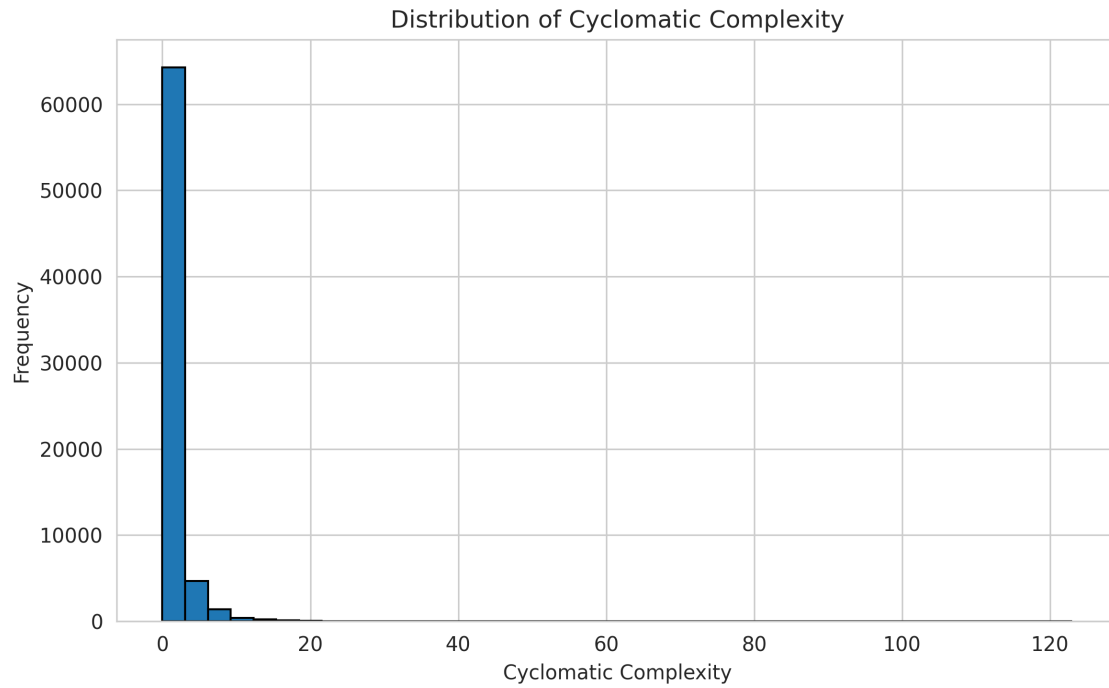


Figure 5. Cyclomatic Complexity Histogram (Long Method Dataset)

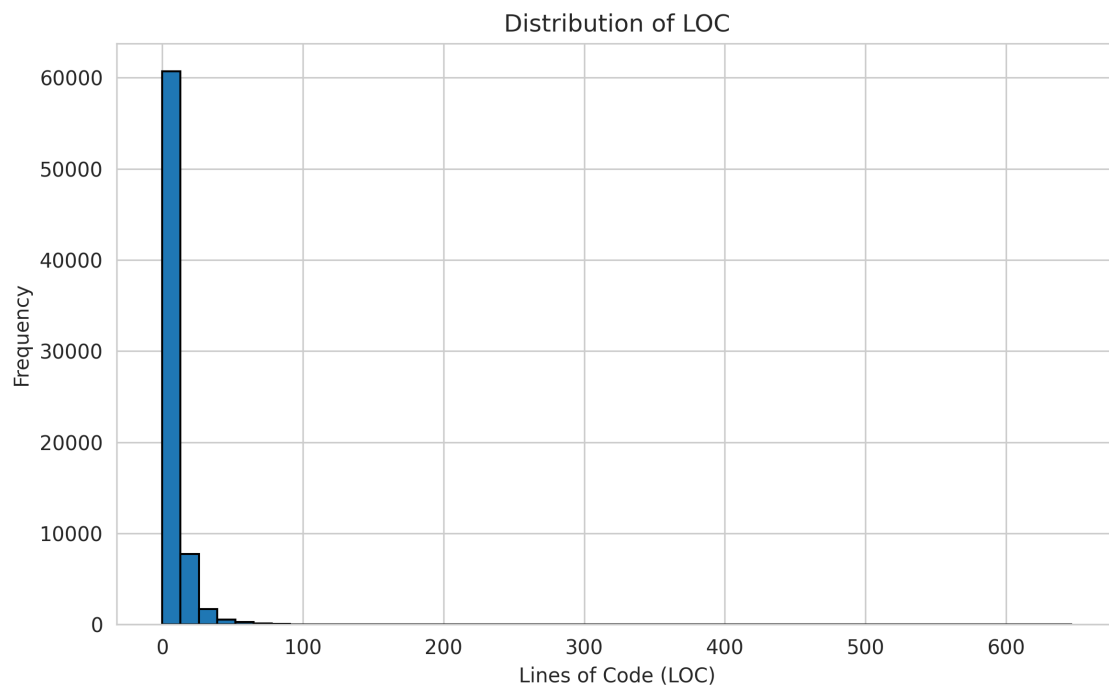


Figure 6. LOC Histogram (Feature Envy Dataset)

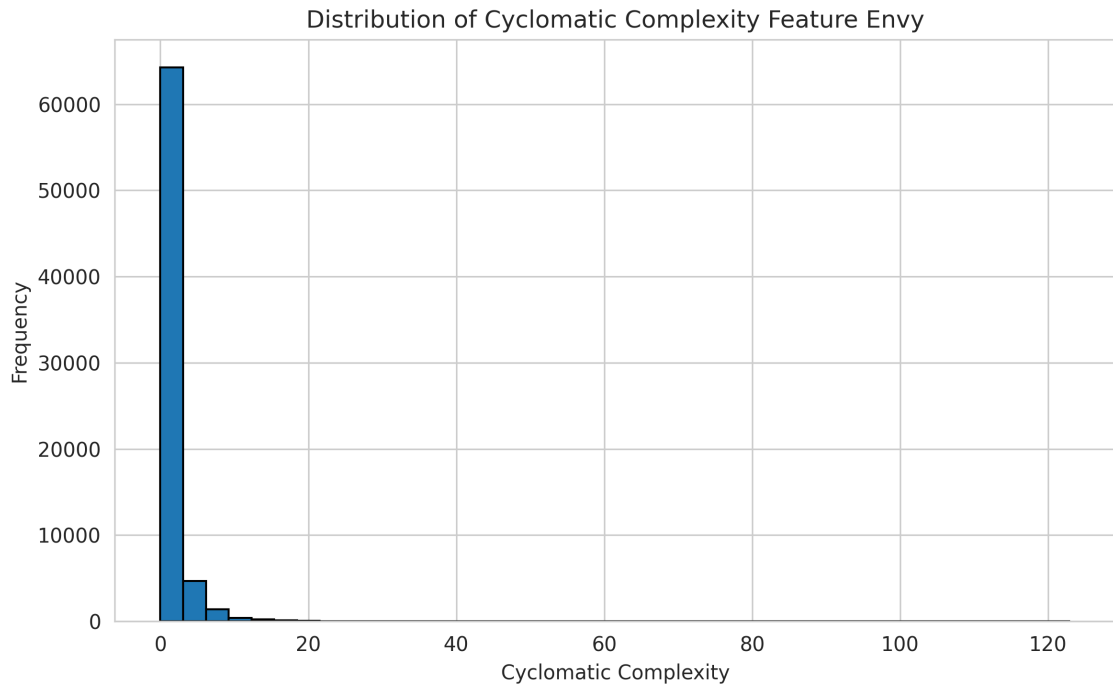


Figure 7. Cyclomatic Complexity Histogram (Feature Envy Dataset)

The LOC and Cyclomatic Complexity histograms for both datasets exhibit highly similar right-skewed distributions. In the Long Method dataset, the mean LOC is 7.73 with a median of 5.0, while the Feature Envy dataset shows a comparable mean LOC of 7.73 with a median of 5.0. Similarly, Cyclomatic Complexity distributions in both datasets show a mean of approximately 1.84 with a median of 1.0. These consistent patterns across both datasets confirm that the underlying metric extraction process produces stable and reproducible results regardless of the code smell labeling applied.

4.4. Correlation Analysis

The correlation analysis reveals that several software metrics are strongly correlated because they measure closely related structural properties. For example, Cyclomatic Complexity and CC exhibit perfect agreement due to equivalent computational definitions. Nevertheless, the majority of representative metrics demonstrate moderate or weak correlations, indicating that they capture complementary aspects of software quality rather than redundant information.

Table 10. Correlation Analysis Between Metric Pairs

Metric Pair	Correlation
CYCLO – CC	1.000
WMC – WMCNAMM	0.999
number_static_methods – number_not_final_static_methods	1.000

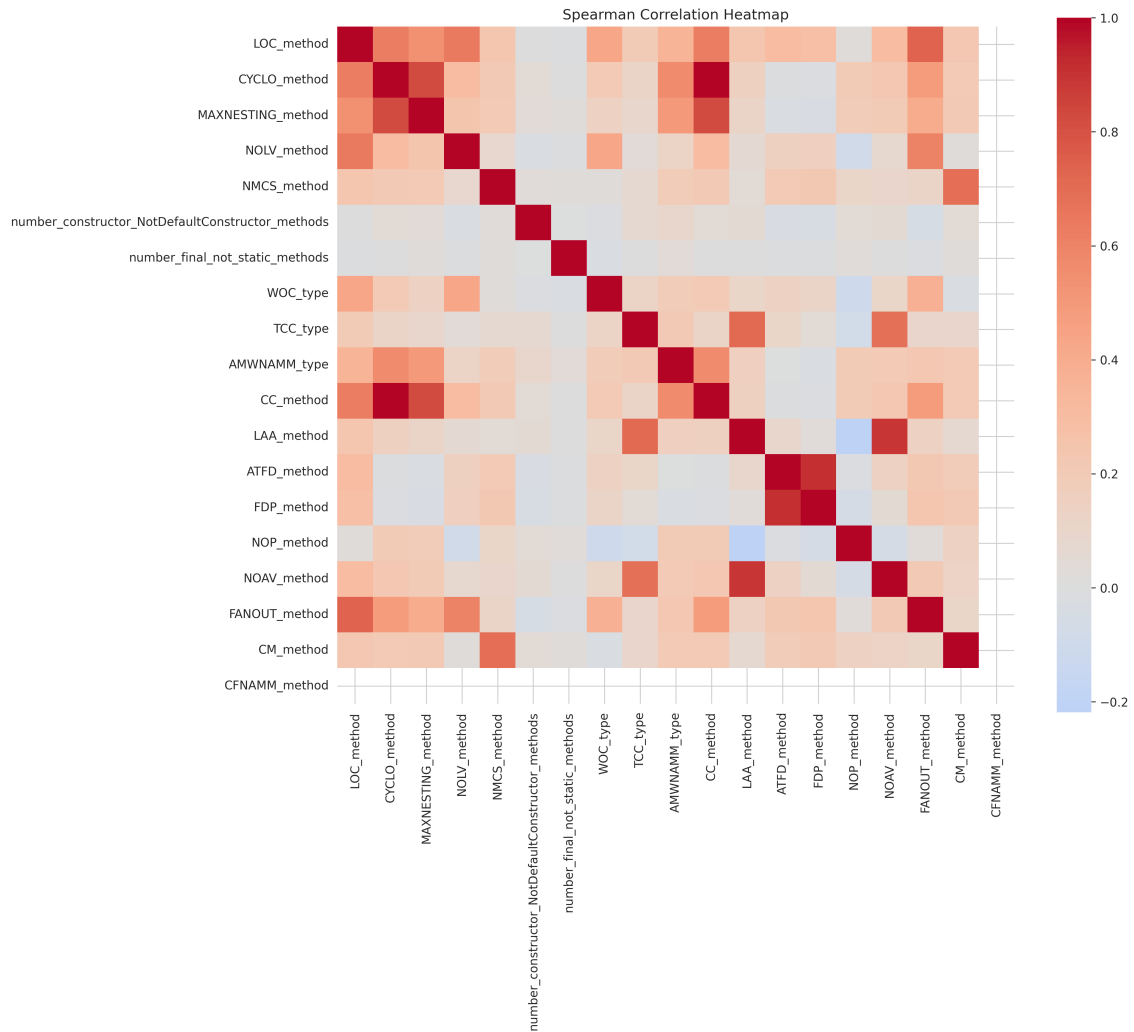


Figure 8. Metric Heatmap

4.5. Class Distribution

After data cleaning, the Long Method and Feature Envy datasets contain the same 71,259 method-level instances. The Long Method dataset contains 232 positive instances (0.33%), whereas the Feature Envy dataset contains 250 positive instances (0.35%). The two labels are not mutually exclusive: 13 method instances are labeled with both code smells.

Table 11. Class Distribution of Long Method and Feature Envy

Class	LM Count	LM %	FE Count	FE %
Non-smell	71,027	99.67%	71,009	99.65%
Smell	232	0.33%	250	0.35%
Total	71,259	100.00%	71,259	100.00%

To further characterize the relationship between the two code smell labels, the common dataset can be partitioned into four mutually exclusive categories: methods without either

smell, Long Method only, Feature Envy only, and methods exhibiting both smells. To examine the co-occurrence of Long Method and Feature Envy, the two labels were analyzed jointly over the same 71,259 method-level observations. The results show that 70,790 methods (99.34%) are labeled as neither Long Method nor Feature Envy. A total of 219 methods (0.31%) are labeled as Long Method only, while 237 methods (0.33%) are labeled as Feature Envy only. Only 13 methods (0.02%) are simultaneously labeled as both Long Method and Feature Envy. These results indicate that the two code smells are largely non-overlapping at the instance level in this dataset. Although both smells represent method-level design problems, their simultaneous occurrence is rare, with only 13 out of 71,259 methods receiving both labels. Consequently, the two labels should not be treated as interchangeable indicators of the same phenomenon. The joint distribution also provides an important basis for subsequent analyses that investigate the relationship between different code smells, particularly when constructing multi-label or multi-smell prediction models.

Table 12. Joint Distribution of Long Method and Feature Envy Labels

Category	Count	Percentage
Neither Long Method nor Feature Envy	70,790	99.34%
Long Method only	219	0.31%
Feature Envy only	237	0.33%
Both Long Method and Feature Envy	13	0.02%
Total	71,259	100.00%

The joint distribution shows that only 13 method instances are shared by the two smell classes. This overlap should be considered when the two labels are analyzed jointly or when machine learning models are evaluated independently for each target.

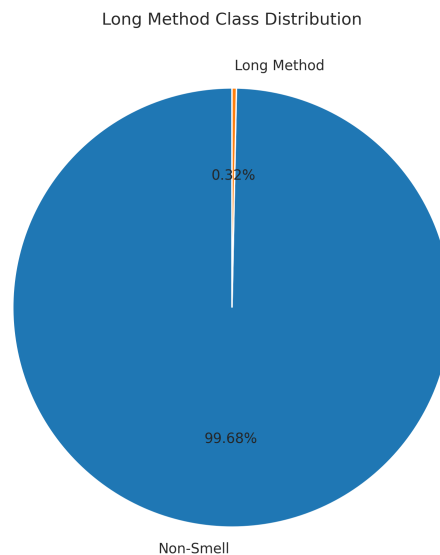


Figure 9. Long Method Class Distribution

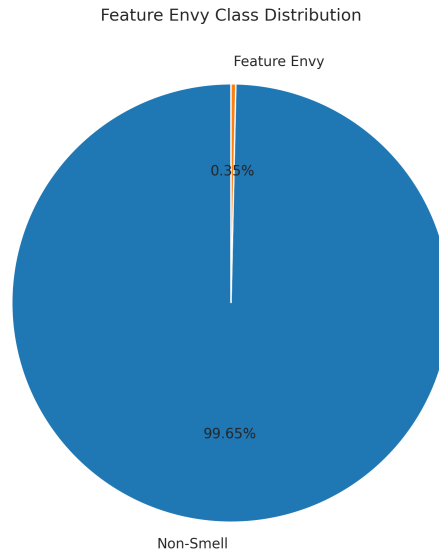


Figure 10. Feature Envy Class Distribution

5. Discussion

5.1. Reliability of the Proposed AST-Based Extraction Pipeline

The verification results demonstrate that the proposed Kopyt-based extraction pipeline is capable of accurately extracting representative software metrics from Kotlin source code. Rather than relying on a single verification approach, this study combines manual source code inspection, independent AST-based verification using the Detekt framework, and algorithm-level verification for metrics requiring inter-class dependency analysis. The three complementary strategies validate different aspects of the extraction process, thereby providing stronger evidence of correctness than a single verification technique alone.

Manual verification confirms that software metrics directly observable from the source code, including software size, logical complexity, nesting structure, local variable usage, parameter count, and method interaction, are computed exactly according to their formal definitions. Meanwhile, metrics involving class cohesion, coupling, inheritance, and design characteristics exhibit consistently high agreement with values produced by the independent AST implementation. The additional algorithm-level verification further confirms that complex metrics such as ATFD, FDP, CM, and CFNAMM correctly implement their mathematical definitions for representative Kotlin methods. Collectively, these results indicate that the proposed extraction pipeline reliably captures both method-level and class-level software characteristics required for constructing machine learning datasets.

Unlike previous dataset construction approaches that typically rely on a single parser or manual validation of a limited number of metrics, the proposed verification framework evaluates representative metrics covering multiple software quality dimensions. Consequently, the reliability of the constructed dataset is supported not only by high agreement values but also by the complementary nature of the adopted verification strategies.

5.2. Quality of the Constructed Dataset

The quality assessment demonstrates that the final datasets satisfy several important characteristics required for empirical software engineering research. The raw Long Method export contained 71,623 records, including 364 malformed records that produced 31,304 apparent missing values because of malformed CSV quoting. Structural inspection of the Feature Envy export identified two additional malformed records. After these records were excluded, the Long Method and Feature Envy datasets were reconciled to the same 71,259 method-level instances, with no missing values or duplicate complete records remaining.

The final dataset contains 89 attributes, including 82 software metrics and seven non-metric attributes consisting of six metadata fields and one code smell label. This distinction is important because the number of dataset attributes should not be interpreted as the number of extracted software metrics.

Descriptive analysis indicates that the extracted software metrics exhibit diverse and right-skewed distributions reflecting the variability of Kotlin source code. Although several representative metrics contain extreme values, these observations correspond to genuinely large or structurally complex methods rather than automatically indicating extraction errors. Such observations should therefore be interpreted in the context of the metric definitions and the underlying source code.

The class distribution is highly imbalanced for both code smells. Long Method represents 0.33% of the final common dataset, while Feature Envy represents 0.35%. This imbalance should be considered when the dataset is used for supervised learning because accuracy alone may be dominated by the majority class. Evaluation measures such as precision, recall, F1-score, and precision–recall analysis are therefore more informative for future classification experiments.

5.3. Characteristics of Representative Software Metrics

Statistical analysis demonstrates that the representative software metrics capture complementary aspects of Kotlin software quality. Metrics such as LOC, Cyclomatic Complexity, and Maximum Nesting Level primarily describe method size and logical complexity, whereas cohesion metrics including TCC and LAA characterize structural relationships among class members. Coupling metrics such as ATFD, FDP, and FANOUT measure interactions with external classes, while class-level metrics including WOC and AMWNAMM describe higher-level design characteristics.

The correlation analysis further indicates that although several complexity-related metrics exhibit moderate positive correlations, many representative metrics remain relatively independent from one another. This observation suggests that the selected metrics describe different structural properties rather than redundant information. Consequently, the feature selection process performed in previous studies successfully identified software metrics that collectively provide comprehensive information for code smell detection.

The existence of only five overlapping metrics between the representative feature sets of Long Method and Feature Envy further supports this observation. Although both code smells share several general software characteristics, each code smell is primarily characterized by distinct software quality dimensions. Long Method emphasizes software size and logical complexity, whereas Feature Envy relies more heavily on coupling and inter-class interaction metrics. This finding explains why different subsets of representative

metrics are required to achieve accurate detection performance for different code smell types.

5.4. Implications for Machine Learning-Based Code Smell Detection

The verified dataset provides a reproducible foundation for future machine learning-based code smell detection research. The final common dataset contains the same 71,259 method-level instances for both Long Method and Feature Envy, while maintaining separate target labels. This structure enables researchers to compare models for the two smell types without introducing differences in the underlying set of analyzed methods.

The severe class imbalance should be explicitly considered in future experiments. Because positive instances represent less than 1% of the final dataset for both smell types, a classifier may achieve high overall accuracy while providing poor detection of positive instances. Consequently, future studies should report class-sensitive evaluation measures and should clearly distinguish between data-level balancing, algorithm-level cost sensitivity, and threshold adjustment.

The dataset also supports a range of machine learning techniques beyond any single classifier used in the labeling procedure, if applicable. The extracted metrics can be utilized by ensemble learning algorithms, gradient boosting models, automated machine learning frameworks, and feature selection techniques. However, future models should be evaluated independently from the procedure used to generate the code smell labels in order to avoid circular evaluation.

Another important implication is the reproducibility of the proposed extraction pipeline. Because the extraction process is implemented directly on Kotlin Abstract Syntax Trees, future researchers can reproduce the dataset construction process, extend the implemented software metrics, or apply the pipeline to additional Kotlin repositories without changing the underlying extraction methodology.

5.5. Comparison with Previous Dataset Construction Approaches

Previous software metric datasets have predominantly focused on Java systems and frequently relied on existing metric extraction tools, parser-based approaches, or manually constructed benchmark datasets. Although these datasets have significantly contributed to software engineering research, many of them do not fully support Kotlin-specific language constructs introduced in modern Android development.

In contrast, the proposed dataset is specifically designed for Kotlin applications and employs a dedicated AST-based extraction pipeline implemented using Kopyt. This implementation enables direct traversal of Kotlin syntax trees while preserving language-specific constructs that may not be accurately handled by language-independent parsers. Moreover, the proposed pipeline simultaneously extracts method-level and class-level software metrics within a unified framework, facilitating the construction of comprehensive datasets for multiple code smell detection tasks.

The additional verification performed using an independent AST implementation further distinguishes this work from previous dataset construction studies. Rather than validating only the resulting datasets, this study explicitly verifies the correctness of the extraction algorithms, thereby increasing confidence in both the extracted software metrics and the constructed dataset.

5.6. Threats to Validity

Several limitations should be considered when interpreting the results of this study. Regarding internal validity, although multiple verification strategies were employed, implementation errors may still exist for software metrics that were not included among the representative verification set. In addition, the data-quality analysis identified malformed CSV records in the exported datasets. These records were removed before the final reconciliation, but their presence indicates that the data-export process itself is a potential source of error and should be controlled in future releases.

Regarding construct validity, the implemented software metrics follow the formal definitions adopted in this study. Manual verification, independent AST-based verification, and algorithm-level validation collectively reduce the possibility that the extracted metrics deviate from their intended definitions. However, the independent verification results were not uniformly identical: one evaluated metric achieved only 68% agreement with Detekt. This discrepancy indicates that some metric definitions or implementation details may require further investigation.

The validity of the code smell labels is another important consideration. The manuscript must explicitly document the actual label-generation mechanism and its provenance. Expert annotation, predefined detection rules, and previously trained machine learning models represent different forms of ground truth and have different implications for reproducibility and validity. Therefore, the final manuscript should use one consistent description of the labeling procedure and report the information required to reproduce it.

With respect to external validity, the constructed dataset consists exclusively of Kotlin open-source projects. Consequently, the reported findings may not be directly generalizable to other programming languages such as Java, Scala, or Swift. Moreover, the Feature Envy positives are concentrated in a limited number of projects: 250 positive instances occur in 12 projects, with 179 instances (71.6% of all Feature Envy positives) originating from a single project. This concentration may introduce project-specific characteristics into the positive class.

Finally, conclusion validity may be affected by the severe class imbalance of both smell labels. Only 232 of the 71,259 final instances are labeled Long Method, while 250 are labeled Feature Envy. Future machine learning studies should therefore use class-sensitive evaluation procedures and, where appropriate, repository-level validation to reduce the risk that project-specific characteristics dominate the reported performance.

6. Conclusions

This study presented an AST-based software metric extraction pipeline for constructing a validated dataset to support Kotlin code smell detection. The proposed approach utilizes the Kopyt framework to traverse Kotlin Abstract Syntax Trees and extract software metrics at the method and class levels. The resulting dataset contains 89 attributes, including 82 software metrics, six metadata attributes, and a code smell label.

Data-quality reconciliation identified malformed records in the raw exports. The initial Long Method export contained 71,623 records, including 364 malformed records associated with 31,304 apparent missing values. The Feature Envy export contained 71,261 records, of which two were identified as malformed during structural inspection. After cleaning and

reconciliation, both code smell datasets contain the same 71,259 method-level instances with no missing values or duplicate complete records.

The final common dataset contains 232 Long Method instances (0.33%) and 250 Feature Envy instances (0.35%). Thirteen method instances are labeled with both code smells, while 219 are Long Method only and 237 are Feature Envy only. These results demonstrate that the two code smell targets can be analyzed on a common set of Kotlin method instances while preserving their distinct labeling characteristics.

To ensure the reliability of the extracted metrics, three complementary verification strategies were employed. Manual source-code inspection was used to validate directly observable software metrics, an independent AST-based implementation using the Detekt framework verified structurally complex metrics, and algorithm-level verification examined coupling-related metric implementations. Most evaluated metrics showed high agreement, although one metric exhibited substantially lower agreement and should be investigated further.

Overall, the proposed pipeline and cleaned dataset provide a reproducible foundation for empirical studies of Kotlin software quality and code smell detection. Future work should expand the repository population, independently validate the code smell labels, investigate the remaining metric discrepancies, and evaluate machine learning models using class-sensitive and repository-aware validation procedures.

Acknowledgment

The research team would like to express their deepest gratitude to Universitas Gadjah Mada for the guidance in writing this article, and also to Universitas Kristen Duta Wacana for the support and funding for this research. We would also like to thank the research team which has all worked well together. This work was supported by the Universitas Kristen Duta Wacana under Study Contract Grand number 266/B.02/Yas/2023.

CRedit authorship contribution statement

A.W.: conceptualization, investigation, reviewing and editing, methodology, validation, visualization, project administration, writing; A.E.P.: investigation, methodology, data analysis, validation, supervision; T.B.A.: conceptualization, methodology, data curation, reviewing, supervision. All authors have read and agreed to the published version of the manuscript.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research was supported by the Universitas Kristen Duta Wacana under Study Contract Grand number 266/B.02/Yas/2023.

Declaration of AI assistance in the writing process

During the preparation of this work the author(s) used OpenAI in order to create formulas in latex and do some modify on that formula format can look more clean. After using this tool/service, the author(s) reviewed and edited the content as necessary and took full responsibility for the content of the published article.

References

- [1] M. Fowler, *Refactoring: Improving the Design of Existing Code*. Addison-Wesley, 1999.
- [2] F. Palomba et al., “A survey on software smells,” *Journal of Systems and Software*, Vol. 135, 2018, pp. 33–55.
- [3] J. Pereira dos Reis, F. Brito e Abreu, G. de Figueiredo Carneiro, and C. Anslow, “Code smells detection and visualization: A systematic literature review,” *Archives of Computational Methods in Engineering*, Vol. 29, No. 1, Mar. 2021, p. 47–94. [Online]. <http://dx.doi.org/10.1007/s11831-021-09566-x>
- [4] P.S. Thakur, S.S. Chouhan, S.S. Rathore, and J. Parmar, “Systematic literature review on software code smell detection approaches,” *Journal of Systems and Software*, Vol. 235, 2026, p. 112784. [Online]. <https://www.sciencedirect.com/science/article/pii/S016412122600018X>
- [5] S. Alodibat and M. Azzeh, “A systematic review on code smell detection approaches in open source projects,” *Software: Practice and Experience*, Vol. 56, No. 7, 2026, pp. 847–877.
- [6] D. Mesbah, N. El Madhoun, K. Al Agha, and A. Zouaoui, “A survey on code smells detection using machine learning techniques,” *Information and Software Technology*, Vol. 198, 2026, p. 108242. [Online]. <https://www.sciencedirect.com/science/article/pii/S0950584926002314>
- [7] A. Wibowo, A.E. Permanasari, and T.B. Adj, “A decade of code smells detection: Evolution, challenges, and optimization strategies,” in *2025 17th International Conference on Information Technology and Electrical Engineering (ICITEE)*, 2025, pp. 1–6.
- [8] L. Madeyski and B. Kitchenham, “Would wider adoption of reproducible research be beneficial for empirical software engineering research?” *Journal of Intelligent and Fuzzy Systems*, Vol. 32, No. 2, 2017, pp. 1509–1521.
- [9] A. Wibowo, A. Erna, and T. Bharata, “Enhanced code smell detection using random forest optimized with particle swarm variants,” *Results in Engineering*, Vol. 29, No. September 2025, 2026, p. 109729.
- [10] G. Bishop, S. Chakraborty, H. Zhou, J. Dehlinger, L. Deng et al., “On-demand generation of high-quality software engineering datasets using large language models and ontologies,” *Automated Software Engineering*, Vol. 33, No. 3, 2026, p. 73. [Online]. <https://doi.org/10.1007/s10515-026-00617-w>
- [11] G. Destefanis, L. Yousefi, M. Shepperd, A. Tucker, S. Swift et al., “An audit of machine learning experiments on software defect prediction,” *Empirical Software Engineering*, Vol. 31, No. 4, 2026.
- [12] H. Zhang and T. Kishi, “Sacs: A code smell dataset using semi-automatic generation approach,” 2026. [Online]. <https://arxiv.org/abs/2602.15342>
- [13] F. Zhang, Z. Zhang, J.W. Keung, X. Tang, Z. Yang et al., “Data preparation for deep learning based code smell detection: A systematic literature review,” *Journal of Systems and Software*, Vol. 216, 2024, p. 112131. [Online]. <https://www.sciencedirect.com/science/article/pii/S0164121224001766>
- [14] T. Lewowski and L. Madeyski, “How far are we from reproducible research on code smell detection? a systematic literature review,” *Information and Software Technology*, Vol. 144, 2022, p. 106783. [Online]. <https://www.sciencedirect.com/science/article/pii/S095058492100224X>
- [15] N. Alomari, A. Alazba, H. Aljamaan, and M. Alshayeb, “SmellyCode++: Multi-label dataset for code smell detection,” *Scientific Data*, Vol. 12, No. 1, jul 2025, p. 1207. [Online]. <https://doi.org/10.1038/s41597-025-05465-z>

- [16] M. Zakeri-Nasrabadi, S. Parsa, E. Esmaili, and F. Palomba, "A systematic literature review on the code smells datasets and validation mechanisms," *ACM Computing Surveys*, Vol. 55, No. 13s, 2023, p. 1–48. [Online]. <http://dx.doi.org/10.1145/3596908>
- [17] P.S. Yadav, R.S. Rao, A. Mishra, and M. Gupta, "Machine learning-based methods for code smell detection: A survey," *Applied Sciences*, Vol. 14, No. 14, 2024. [Online]. <https://www.mdpi.com/2076-3417/14/14/6149>
- [18] H.M. Alhadeaf and M. Alrashoud, "A unified multi-label code smell dataset for code smell detection at different granularities," *IEEE Access*, Vol. 14, 2026, pp. 714–737.
- [19] L. Madeyski and T. Lewowski, "Mlcq: Industry-relevant code smell data set," Feb. 2020. [Online]. <https://doi.org/10.5281/zenodo.3666840>
- [20] Y. Zhang, C. Ge, H. Liu, and K. Zheng, "Code smell detection based on supervised learning models: A survey," *Neurocomputing*, Vol. 565, 2024, p. 127014. [Online]. <https://www.sciencedirect.com/science/article/pii/S0925231223011372>
- [21] K. Contributors, "kopyt: A python library available on pypi," <https://pypi.org/project/kopyt/>, 2024, accessed: 2026-07-26.
- [22] M. Akhin, M. Belyaev et al., *Kotlin Language Specification: Control- and Data-Flow Analysis*, JetBrains / JetBrains Research, 2020, accessed: 2026-07-26. [Online]. <https://kotlinlang.org/spec/control--and-data-flow-analysis.html>
- [23] A. Chen, Y. Yan, and D. Poshyvanyk, "Acer: An ast-based call graph generator framework," 2023. [Online]. <https://arxiv.org/abs/2308.15669>
- [24] A. Wibowo, A.E. Permanasari, T.B. Adji, and A. Wibowo, "Custom rule-based feature engineering for code smell detection in kotlin using detekt," in *2025 10th International Conference on Information Technology and Digital Application (ICITDA)*, 2025, pp. 1–8.
- [25] S. Dewangan, R.S. Rao, A. Mishra, and M. Gupta, "Code smell detection using ensemble machine learning algorithms," *Applied Sciences*, Vol. 12, No. 20, 2022. [Online]. <https://www.mdpi.com/2076-3417/12/20/10321>
- [26] H. Liu, J. Jin, Z. Xu, Y. Zou, Y. Bu et al., "Deep Learning Based Code Smell Detection," *IEEE Transactions on Software Engineering*, Vol. 47, No. 09, Sep. 2021, pp. 1811–1837. [Online]. <https://doi.ieeecomputersociety.org/10.1109/TSE.2019.2936376>
- [27] M. Abdelkader, "A novel approach for enhancing code smell detection using random convolutional kernel transform," *e-Informatica Software Engineering Journal*, Vol. 19, 2025, p. 250106.
- [28] F. Arcelli Fontana and M. Zaroni, "Code smell severity classification using machine learning techniques," *Know.-Based Syst.*, Vol. 128, No. C, Jul. 2017, p. 43–58. [Online]. <https://doi.org/10.1016/j.knosys.2017.04.014>
- [29] M. Mhawish and M. Gupta, "Predicting code smells and analysis of predictions: Using machine learning techniques and software metrics," *Journal of Computer Science and Technology*, Vol. 35, 11 2020, pp. 1428–1445.
- [30] S. Zellagui, A. Kenniche, S. Fadloun, and S. Meshoul, "Leveraging code language models for code smells detection," *Applied Sciences*, Vol. 16, No. 14, 2026. [Online]. <https://www.mdpi.com/2076-3417/16/14/7040>
- [31] S. Dewangan, R.S. Rao, S. Chowdhuri, and M. Gupta, "Severity classification of code smells using machine-learning methods," *SN Computer Science*, Vol. 4, 07 2023.
- [32] I.S.R. Ugli, J. Cheon, and G. Woo, "Code smells and development efficiency in native and cross-platform mobile applications: A study of java, kotlin, and flutter," *Journal of Information Processing Systems*, Vol. 21, No. 4, 2025, pp. 401–412.
- [33] S. Prokić, N. Luburić, J. Slivka, and A. Kovačević, "Prescriptive procedure for manual code smell annotation," *Science of Computer Programming*, Vol. 238, 2024, p. 103168. [Online]. <https://www.sciencedirect.com/science/article/pii/S0167642324000911>
- [34] J. Slivka, N. Luburić, S. Prokić, K.G. Grujić, A. Kovačević et al., "Towards a systematic approach to manual annotation of code smells," *Science of Computer Programming*, Vol. 230, 2023, p. 102999. [Online]. <https://www.sciencedirect.com/science/article/pii/S0167642323000813>
- [35] R. Gupta, J.K. Seth, A. Sharma, and A. Goyal, "Can Public Code Smells Datasets Be Trusted?" *Journal of Communications Software and Systems*, Vol. 21, No. 4, 2025, pp. 477–488.

- [36] E. Tempero, C. Anslow, J. Dietrich, T. Han, J. Li et al., “The qualitas corpus: A curated collection of java code for empirical studies,” 11 2010, pp. 336–345.
- [37] R. Terra, L. Miranda, M. Valente, and R. Bigonha, “Qualitas.class corpus: A compiled version of the qualitas corpus,” *ACM SIGSOFT Software Engineering Notes*, Vol. 38, 08 2013, pp. 1–4.
- [38] A. Santana, E. Figueiredo, J. Alves Pereira, and A. Garcia, “An exploratory evaluation of code smell agglomerations,” *Software Quality Journal*, Vol. 32, No. 4, 2024, pp. 1375–1412. [Online]. <https://doi.org/10.1007/s11219-024-09680-6>
- [39] J.M. González-Barahona and G. Robles, “On the reproducibility of empirical software engineering studies based on data retrieved from development repositories,” *Empirical Software Engineering*, Vol. 17, No. 1-2, 2012, pp. 75–89.
- [40] J.M. Gonzalez-Barahona and G. Robles, “Revisiting the reproducibility of empirical software engineering studies based on data retrieved from development repositories,” *Information and Software Technology*, Vol. 164, No. August, 2023, p. 107318. [Online]. <https://doi.org/10.1016/j.infsof.2023.107318>
- [41] R. Lefeuve, J. Galasso, B. Combemale, H. Sahraoui, and S. Zacchiroli, “Fingerprinting and building large reproducible datasets,” 2023. [Online]. <https://arxiv.org/abs/2306.11391>
- [42] M.F. Bosu and S.G. MacDonell, “Data quality in empirical software engineering: a targeted review,” in *Proceedings of the 17th International Conference on Evaluation and Assessment in Software Engineering*, EASE '13. ACM, Apr. 2013, p. 171–176. [Online]. <http://dx.doi.org/10.1145/2460999.2461024>

Authors and affiliations

Argo Wibowo
e-mail: argowibowo528951@mail.ugm.ac.id,
argo@staff.ukdw.ac.id
ORCID: <https://orcid.org/0009-0000-2689-5461>
Department of Electrical and Information
Engineering, Universitas Gadjah Mada, Indonesia,
Department of Information System, Universitas
Kristen Duta Wacana, Indonesia

Adhistya Erna Permanasari
e-mail: adhistya@ugm.ac.id
ORCID: <https://orcid.org/0000-0002-2663-4074>
Department of Electrical and Information
Engineering, Universitas Gadjah Mada, Indonesia

Teguh Bharata Adji
e-mail: adji@ugm.ac.id
ORCID: <https://orcid.org/0000-0001-7856-1498>
Department of Electrical and Information
Engineering, Universitas Gadjah Mada, Indonesia